

# Evaluación técnica

## I. Casos de uso seleccionados e implementados.

Se añadió un sensor y una bomba conectada a un relé para el manejo seguro del encendido y apagado de la bomba. El sensor MAX6675 es una termocupla que mide la temperatura del agua y se conecta al programa para que el usuario pueda saber la temperatura del agua y en un futuro encender o apagar el calentador automáticamente. Seguidamente el relé y la bomba permiten al programa encender la bomba cuando el nivel del agua es muy bajo y apagarla cuando el nivel es suficiente. Se añadieron finalmente leds para confirmación de funcionamiento correcto, manejo de errores y alertas, temperatura del agua y nivel del agua.

## II. Patrones Implementados

### Patrón Command

**Problema Actual:** La interfaz de usuario (Console que implementa UI) o el controlador principal estaría fuertemente acoplado a la lógica de negocio y a los dispositivos. Si el usuario ingresa una acción, el sistema usaría múltiples condicionales para decidir qué ejecutar. Añadir nuevas acciones requeriría modificar esta clase central constantemente.

El patrón ayuda a implementar reglas como **Bajo Acoplamiento, OCP, Alta Cohesión.**

- Interfaz común Command que define un contrato único mediante el método execute(context: AppContext).
- CommandHandler actúa como Invocador: mantiene un HashMap<String, Command> de comandos registrados y ejecuta el comando correspondiente a la entrada sin conocer los detalles de la operación.
- Comandos concretos (LoginCommand, PumpOnCommand, PumpOffCommand, ViewLogsCommand, ViewLevelCommand) encapsulan la solicitud. Cada uno conoce a su Receptor específico (ej. IPump para las bombas o interfaces de estado) para llevar a cabo la lógica.

### Patrón Observer

**Problema Actual:** Solo WaterLevelManager reacciona a eventos de los sensores (ISensor / Sensor). La clase Alarm no está integrada de forma reactiva, Console (UI) no recibe notificaciones automáticas para actualizar la pantalla.

El patrón ayuda a implementar reglas como **Bajo Acoplamiento, Alta Cohesión.**

- WaterLevelManager actúa como el Sujeto (Publisher/Subject). Es el encargado de mantener una lista de dependencias y notificarles cuando el estado del nivel del agua cambia.
- Console (implementando UI) se suscribe para recibir actualizaciones automáticas y refrescar la información en pantalla sin necesidad de consultar repetidamente.

## Patrón Strategy

**Problema Actual:** WaterLevelManager tenía los umbrales de encendido y apagado de la bomba escritos directamente en el código (hardcodeados). Esto impedía cambiar el comportamiento de control de la bomba sin modificar la clase, violando OCP. No había forma de adaptar el sistema a distintas condiciones de uso (ahorro de energía, uso intensivo) sin reescribir lógica interna.

El patrón ayuda a implementar reglas como OCP, Bajo Acoplamiento, Alta Cohesión, Polimorfismo.

- Interfaz común IPumpStrategy que define el contrato mediante los métodos shouldTurnOn(int level) y shouldTurnOff(int level). Ninguna clase de alto nivel conoce la estrategia concreta.
- ConservativeStrategy encapsula una política de ahorro: enciende la bomba cuando el nivel baja al 10% y la apaga al alcanzar el 80%.
- EfficientStrategy encapsula una política de uso intensivo: enciende al 20% y apaga al 90%, priorizando disponibilidad de agua.
- WaterLevelManager actúa como contexto: recibe cualquier IPumpStrategy por constructor e invoca strategy.shouldTurnOn() / strategy.shouldTurnOff() en lugar de evaluar umbrales fijos. Añadir una nueva estrategia no requiere modificar esta clase.
- - Main inyecta new ConservativeStrategy() al construir el manager, centralizando la decisión de qué política usar en el punto de entrada de la aplicación.

## III. Clases Cambiadas O Creadas:

- **Sensor (Jerarquía):** Se eliminó la duplicación de lógica en **handleSensor()**. Ahora es un método concreto en la clase base que orquesta la ejecución, delegando los detalles específicos a métodos abstractos (getSensorType, getSensorDetail).
- **EventHandler:** Se sustituyó la lista interna por una CopyOnWriteArrayList. Esto garantiza **Thread Safety** en el código, evitando excepciones al notificar eventos en entornos concurrentes.
- **RepositoryLog & ViewLogsCommand:** Se encapsuló el acceso al sistema de archivos. El comando ya no conoce la ruta ni la lógica de lectura; simplemente solicita los datos al repositorio, respetando la integridad de la capa de infraestructura.
- **LoginCommand:** Se extrajo el Scanner (Input). Ahora el comando recibe los datos procesados, desacoplándose totalmente de la consola de Java.

## Algunos patrones de diseño desarrollados

- **Template (Sensor):** Centraliza el flujo de procesamiento de señales de hardware, obligando a los sensores nuevos a seguir la misma estructura.
- **Strategy (PasswordHasher):** Permite intercambiar el algoritmo de cifrado (ej. de Simple a BCrypt) sin tocar una sola línea del AuthenticationService.
- **Adapter (ArduinoSerial):** Actúa como traductor entre el protocolo físico de hardware y la lógica de negocio del dominio.
- **Facade (AppContext):** Unifica el acceso a servicios complejos, sirviendo como punto de entrada simplificado para la capa de aplicación.

## ¿Por Qué Mejoró el Diseño?

El diseño mejoró con creces pues pasó a ser más flexible y modular, pese a algunos problemas con sobre-implementación y falta de “clases de acceso global” o algo similar que se requieren en bastantes de las clases como lo es RepositoryLog y el sistema de eventos.

Tomamos como enfoque principal la modularidad, facilidad de modificación y abstracción, resultando en la facilidad de crear pruebas usando clases concretas de prueba para clases abstractas. También, esta modularidad permite que los cambios o clases añadidas no afecten otras clases en otras capas, que facilita la extensibilidad de código.

## Conexión con principios SOLID y GRASP

Aunque el proyecto cumple en mayor medida con los principios SOLID y GRASP, es pertinente destacar en los que se enfocó el proyecto mayormente, haciendo un fuerte énfasis en no romper estas reglas en ninguna parte del código o diagrama.

### Principios SOLID

- **SRP (Responsabilidad Única):** Evidente en la separación de sensores, controladores, eventos, managers y demas, como ejemplo TemperatureSensor solo se encarga de la captura de datos y la lógica de reacción reside en el Manager correspondiente.
- **OCP (Abierto/Cerrado):** El uso del patrón **Command** permite añadir nuevas funcionalidades (ej. ResetCommand) simplemente creando una nueva clase, sin modificar el CommandHandler.
- **DIP (Inversión de Dependencias):** Las clases de alto nivel (en capa de Servicios) dependen de abstracciones (UserRepository, PasswordHasher) y no de implementaciones concretas de infraestructura.

### Principios GRASP

- **Bajo Acoplamiento / Alta Cohesión:** Al usar un bus de eventos (EventHandler), los componentes de hardware y software no se conocen entre sí. Solo conocen el "Evento", lo que permite módulos altamente enfocados y desacoplados.
- **Polimorfismo:** Se utiliza para manejar los distintos tipos de sensores y comandos mediante una interfaz común, eliminando estructuras complejas de if/else o switch en el código principal.
- **Fabricación Pura (Pure Fabrication):** Clases como ApplicationContext y Repositories no representan objetos del mundo real ("Tanque"), sino que son invenciones de diseño para mantener la limpieza y la gestión del estado del sistema.